Verification plan & Subroutines – Extra

1)

Label	Description	Stimulus Generation	Functional Coverage (Later)	Functionality Check
DFF_1	When the reset is asserted, the output q must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
DFF_2	Tyring all possible valid values of inputs	Directed during the simulation	-	A checker in the testbench comparing the DUT output with the golden model output

Bug: missing begin-end in the if-condition for USE_EN

```
module dff(clk, rst, d, q, en);
parameter USE_EN = 1;
input clk, rst, d, en;
output reg q;

always @(posedge clk) begin
   if (rst)
      q <= 0;
   else
      if(USE_EN) begin
         if (en)
            q <= d;
      end
      else
         q <= d;
end

endmodule
```

t1:

```
module dff_t1_tb;
   logic clk, rst, d, en;
   logic q_exp, q_dut;

   localparam USE_EN = 1;

   integer error_count = 0;
   integer correct_count = 0;

   dff #(.USE_EN(USE_EN)) DUT(.clk(clk), .rst(rst), .d(d), .q(q_dut), .en(en));

   // Create the clock and rst
   initial begin
      clk= 0;
      forever
         #1 clk= ~clk;
   end

   initial begin
      rst = 1; d = 0; en = 0; check_result;
      rst = 1; d = 0; en = 1; check_result;
      rst = 1; d = 1; en = 0; check_result;
      rst = 1; d = 1; en = 1; check_result;
      rst = 0; d = 0; en = 0; check_result;
      rst = 0; d = 0; en = 1; check_result;
      rst = 0; d = 1; en = 0; check_result;
      rst = 0; d = 1; en = 1; check_result;

      rst = 1; check_result;

      $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
      $stop;
   end

   //Checker
   task check_result;
      @(negedge clk);
```

```
    //Checker
    task check_result;
       @(negedge clk);
       if (q_exp !== q_dut) begin
         error_count = error_count + 1;
         $display("%t: Error: For rst = 0d%0d, en=0d%0d, and d=0d%0d, q should equal 0d%0d but is 0d%0d ", $time, rst, en, d, q_exp, q_dut);
       end
       else
         correct_count = correct_count + 1;
    endtask

    //Golden model
    always @(posedge clk) begin
       if(rst)
           q_exp <= 0;
       else
          if (en)
             q_exp <= d;
    end
 endmodule
```

t2:

```
module dff_t2_tb;
   logic clk, rst, d;
   logic q_exp, q_dut;

   localparam USE_EN = 0;

   integer error_count = 0;
   integer correct_count = 0;

   dff #(.USE_EN(USE_EN)) DUT(.clk(clk), .rst(rst), .d(d), .q(q_dut), .en());

   // Create the clock and rst
   initial begin
      clk= 0;
      forever
         #1 clk= ~clk;
   end

   initial begin
      rst = 1; d = 0; check_result;
      rst = 1; d = 0; check_result;
      rst = 1; d = 1; check_result;
      rst = 1; d = 1; check_result;
      rst = 0; d = 0; check_result;
      rst = 0; d = 0; check_result;
      rst = 0; d = 1; check_result;
      rst = 0; d = 1; check_result;

      rst = 1; check_result;

      $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
      $stop;
   end

   //Checker
   task check_result;
      @(negedge clk);
```

Karim Waseem

```
   //Checker
   task check_result;
      @(negedge clk);
      if (q_exp !== q_dut) begin
        error_count = error_count + 1;
        $display("%t: Error: For rst = 0d%0d, and d=0d%0d, q should equal 0d%0d but is 0d%0d ", $time, rst, d, q_exp, q_dut);
      end
      else
        correct_count = correct_count + 1;
   endtask

   //Golden model
   always @(posedge clk) begin
      if(rst)
         q_exp <= 0;
      else
         q_exp <= d;
   end
endmodule
```

```
vlib work
vlog dff.v dff_t1_tb.sv dff_t2_tb.sv +cover
vsim -voptargs=+acc work.dff_t1_tb -cover
coverage save dff_t1.ucdb -onexit -du dff
run -all
quit -sim
vsim -voptargs=+acc work.dff_t2_tb -cover
coverage save dff_t2.ucdb -onexit -du dff
run -all
quit -sim
vcover merge dff_merged.ucdb dff_t1.ucdb dff_t2.ucdb -du dff
vcover report dff_merged.ucdb -details -annotate -all -output code_cvr.txt
```

Kareem Waseem